

Introduction

For this project my initial idea was to build something that i would actually want to use myself. I didn't want to just make a simple demo that shows my understanding of this module i wanted to build something personal that i would be proud to run. The main idea that i had from the beginning was to build my own perfect virtual chill spot ultimately being a private rooftop at sunset where i could listen to music i love, then also have the option to visit my favourite cartoon characters as well. This is what started my XR Rooftop project.

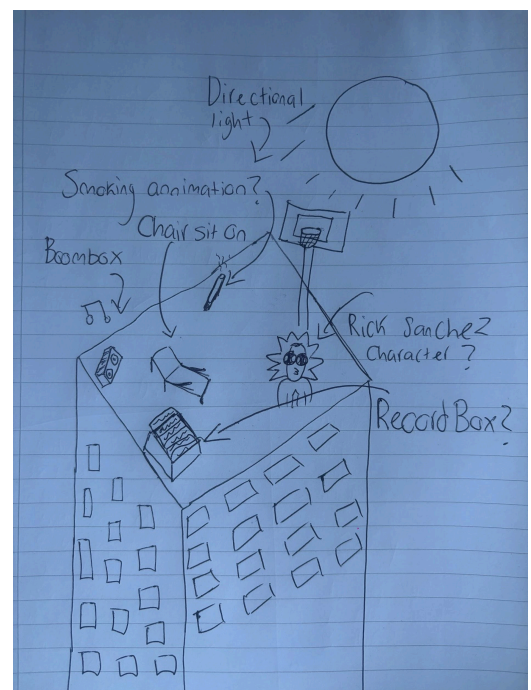
It is a browser based 3D interactive application built with Javascript, [Three.js](#) and partially the WebXR API. the experience so far spans across three connected scenes which are a music filled rooftop for chilling out and listening to my favourite songs, the option to drive to a bar where you meet the character Rick Sanchez, and then later teleport to a model of his garage where other characters have been added. This project is aimed at young adults and students who are into music, gaming, cartoons and interactive media, also with VR users on headsets like the oculus quest as a secondary audience. The character implementation, record box system and hip hop music album choices were all deliberate choices for this specific audience rather than random selections.

Idea Generation

The starting point for this project came from more of a feeling rather than a technical plan. I thought about what kind of XR and VR experience i would actually sit down and use and also be able to show my friends so that they could use this experience too and the idea i landed on was to build my own private relaxing world. This idea came from Rick and Morty "The Old Man and the Seat" (Season 4, Episode 2)" where the character Rick Sanchez builds his own private world to go to the toilet in peace. This idea then grew into the rooftop concept. From there I sketched out a rough scene flow where you could have your own designated spot for relaxing and then also have options to visit other environments with characters and dialogue. Having this set idea mapped out early helped keep the project focused for me.

From this initial concept I then began to think more about how the experience would function as a user so i wanted the rooftop to feel more interactive rather than static, which led to the idea of building a music system where user could physically select and play tracks using object within the scene. This then developed into the record box and boombox system which allows the users to engage with the environment in a more useful way.

After i began building this i decided early on that i didnt want this experience to be limited to just as single scene as i was enjoying building the



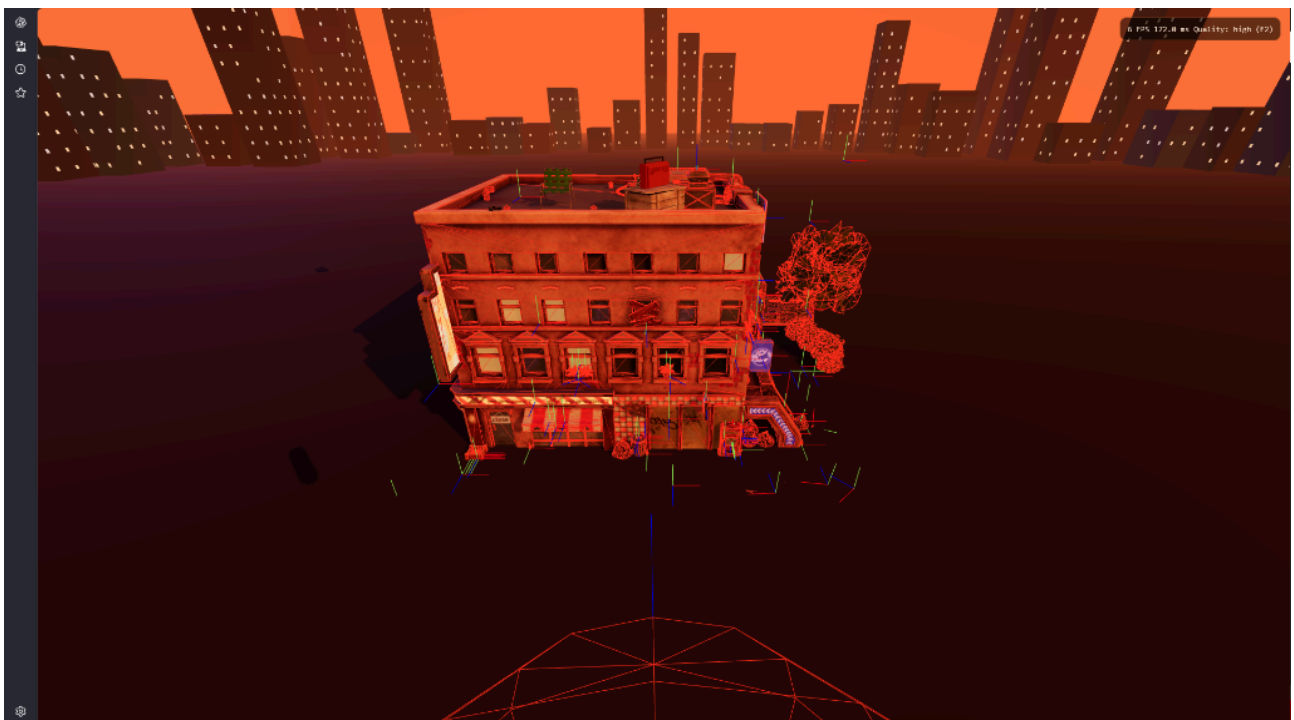
environments and already had lots of time to spare to build on the project further. To make the project more engaging i decided to introduce progression between scenes this is where the idea of travelling from the rooftop to another location came from, which then later became the van interaction and transition into the bar scene.

3D Assets

All of the models that i used in my project were sourced from Sketchfab and downloaded in GLB format. I always chose GLB over formats like FBX because GLB bundles things like geometry, materials and textures all into a single file which makes [Three.js](#) imports way more simple. Using FBX would require managing separate texture files so would be a weaker

choice. In my project every model goes through the same `JS modelLoader.js` function which handles positioning, scaling, basic setup and animation playback automatically.

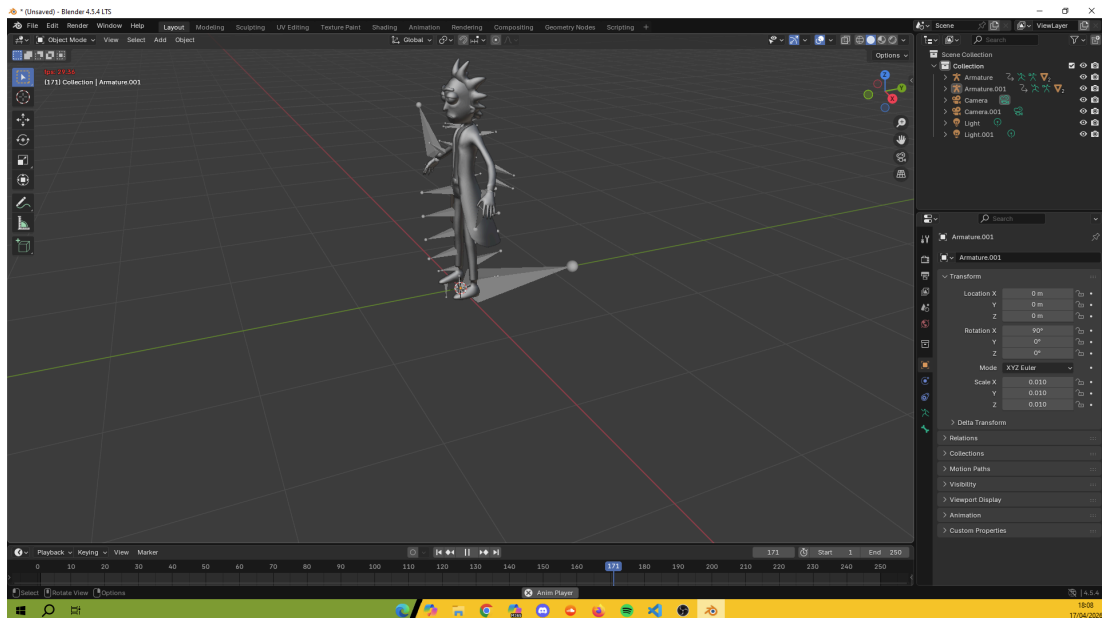
For any model that needs collision detection, i added physics to it so that the player could properly walk on and interact with the environment. Instead of using simple shapes like boxes, i used collision that actually matches the shape of the model this is called “concave”. This was important for more or less all of my environments like the rooftop, bar and garage scenes as they have more complex structures that would not feel accurate with basic collision as in the bar scene for example you need to walk up and down stairs which wouldn't work well with basic collision.



To achieve this i had to loop through each part of the model and apply a physics body to it. These physics bodies are invisible in the actual play game and are only used for collision meaning they do not affect how the model looks at all, however when i change this piece of

code to true `const DEBUG_LOG_MOVEMENT = false;` It enables me to go into “developer mode” so that i can see the meshes of all my models clearly with [ammo.js](#) like in the image above. Static objects in the environment are also set so they don't move which improves the overall projects performance.

Towards the end of my project i also attempted to create an animated version of the Rick Sanchez model in blender. I spent lots of time learning rigging and animation using the NLA editor, however when exporting the model into Three.js i kept on encountering issues where the mesh would deform or the animations would not load correctly. I spent a lot of time on trying to implement this and it caused a setback for my project overall towards the deadline. Although i was unable to use my own animated model in the final project, this process helped me understand how 3D models, animations and file formats work together in a pipeline and did help me build my skills in blender having spent so much time using it.



Development Process

Environment

The first thing i built in this project was the base scene in `JS createScene.js` where i setup the lighting and overall look of the environment i had pictured. To do this i used tone mapping and specific colour settings to make everything look more realistic and less flat and boring. The skybox that i made basically at the very beginning of the project is something i was very happy with as i havent changed it once since it was implemented. I at first attempted to use textures for the floor and hdr models for the sky like i was taught in class, however they just weren't giving off the look that i was hoping for the project. Instead of using a hdr model i created a large sphere around the scene myself and applied colours to it so they would blend smoothly giving a really relaxing orange sunset colour which was exactly what i was looking for.

```
const skyGeo = new THREE.SphereGeometry(450, 32, 32);
const skyMat = new THREE.MeshBasicMaterial({
  vertexColors: true,
  side: THREE.BackSide, // render the inside of the sphere, not the outside
});
const topColour = new THREE.Color(0x1a0533); // deep purple at the top
const midColour = new THREE.Color(0xff6b35); // warm orange in the middle
const horizonColour = new THREE.Color(0xff9d7a); // soft pink/peach at horizon
const positions = skyGeo.attributes.position;
const colors = [];

for (let i = 0; i < positions.count; i++) {
  const y = positions.getY(i);
  const t = (y + 450) / 900;

  let colour = new THREE.Color();

  if (t > 0.5) {
    colour.lerpColors(midColour, topColour, (t - 0.5) * 2);
  } else {
    colour.lerpColors(horizonColour, midColour, t * 2);
  }

  colors.push(colour.r, colour.g, colour.b);
}

skyGeo.setAttribute("color", new THREE.BufferAttribute(
  new Float32Array(colors), 3
));
```

The code here is used to implement the sky gradient loops through every point on the sphere and changes its colour based on its height; this then creates a smooth gradient effect. I used 3 main colours as you can see in the code: dark purple at the top, a warm orange in the middle and a peach colour near the horizon. These colours are blended together so they gradually fade into each other which gives the sky its natural sunset look. The sphere also has to be rendered from the inside so that the player is surrounded by it to create a more immersive environment.

Physics and Player

Early on in the project I used a basic collision system with hardcoded boundary checks. The way I implemented this was by placing an invisible floor on top of my rooftop model so that the player could stand on it. However, I quickly realised this approach had major issues, as the player could still walk through walls and fall through parts of the environment. This made the experience feel unrealistic and unpolished.

To improve this and add real physics and collision I replaced the system with a proper physics implementation using [ammo.js](#). I created a capsule-shaped collider for the player in [playersetup.js](#). A capsule was chosen just because it moves smoothly across uneven surfaces and edges, unlike a box collider which can easily get stuck in places or just behave

unpredictably. This was really important for my environments such as the bar scene which has stairs you need to be able to walk up and down and also with objects on the rooftop model that would look unrealistic if you couldn't walk over them.

The players setup supports multiple movement states including walking, sprinting, jumping, crouching and sitting. The sitting system was put in place for interactions such as the chair on the rooftop. When the player is sitting their velocity is set to 0 each frame so that it prevents the physics system from pushing you out of position. Overall integrating [ammo.js](#) was a significant improvement to the whole project and improved the realism and usability as well as being a completely necessary implementation for my project to work as intended.

Interaction and prompt fix

All of the interactive objects in the project use the same raycasting system that are defined in `JS createControls.js`. A ray is cast from the centre point of the camera and then when it intersects with an objects invisible interaction box within a certain distance then a prompt appears on screen telling the user what they can do (e.g “press E to talk to Rick)

In the beginning of the rooftop scene development I encountered a problem where multiple prompts would show up at the same time, this made everything look messy, confusing and also caused lag. To fix this i created the `JS createPrompt.js` file as a manager to control all of the prompts in the project as i knew that i would have a lot. Instead of having each object handle its own UI for the prompt, they instead register with the [createprompt.js](#) system and are given a priority value. This basically means that only the most important prompt shows at any one time which fixed the issue and made things much cleaner for the user.

I also ran into a performance issue where the game would severely lag when looking at certain objects, especially the record box. This was because the raycast checks were running every frame. To fix this, I added a simple throttle so the check would only run every 80 ms instead. This stopped the raycast looping over and over for checks every frame and removed the lag while still keeping the game feeling responsive.



Audio System and record box implementation

The music system for this project was the part which i was most interested in building and spent most of my time implementing and making sure that it felt interactive rather than just background audio. I wanted the music integration to be a big part of the game and one of the main features, so i spent lots of time thinking about design options for implementing music into my rooftop environment and learning lots about css to help me create a professional looking user interface.

I used [Three.js](#) audio classes to attach sound directly to the boombox object on the rooftop environment which means that the music changes depending on where the player is standing in the scene. If you move further away from the boombox model the volume will drop off sounding like its actually coming from the distance, if you turn your head away the sound will also shift direction between position. This makes it feel like the music is actually coming from the boombox rather than just making it play through both ears in the background, i think that this makes the environment way more immersive for the user and fits in perfectly with the relaxation vibe i was going for. You can see in the code below where sound is added to the boombox and given a distance value.

```
const speakerSound = new THREE.PositionalAudio(listener);
speakerSound.setRefDistance(5);
speakerSound.setLoop(false);
speakerSound.setVolume(1.0);
if (boombox) boombox.add(speakerSound);
```

I also added a second audio type using `THREE.Audio` for when the player puts on the headphones model found on the rooftop. This then removes the positional effect and plays the music evenly which makes it feel like you are wearing headphones.

```
const headphoneSound = new THREE.Audio(listener);
headphoneSound.setLoop(false);
headphoneSound.setVolume(1.0);
```

Switching between these two happens automatically depending on whether the player is wearing headphones this made the system feel way more realistic. Another part that i added later was making the boombox react to the music visually in the rooftop scene. To do this i used `THREE.AudioAnalyser` to read the audio levels and then used that data to control a small glow lighting from the speakers.

```
const speakerAnalyser = new THREE.AudioAnalyser(speakerSound, 64);
```

This value is then updated every frame and also used to increase the brightness of materials and light on the boombox and other models around it which makes it pulse in the time of the music.

The record selection part i built is handled in `JS createRecordBox.js` where i built a full UI using HTML and CSS handled in the `# recordBox.css` file. When the player is looking at the model and has the prompt to press E it opens up a UI in the centre of the screen showing album artwork, track names, and navigation



controls which i learnt out how to implement through documentation such as stackOverflow. This UI is all created in my code by using elements such as:

```
const ui = document.createElement("div");
ui.className = "record-box-ui";
document.body.appendChild(ui);
```

I also added controls for browsing the different song options, selecting them and displaying whatever your current selection is. The player can scroll through tracks using arrows or

dragging the mouse, and the UI using the `function updateDisplay()`.

One of my favourite features that I had the idea of implementing is the locked tracks feature. Some songs that are in the recordbox UI are made hidden unless the correct password is entered, the tracks that require a password are some rap songs that me and my friends created in highschool for fun as a joke, the reason i wanted to implement this is because i thought it would be a funny implementation if my friends decided to play my game too. The way i set this up was using environment variables at the top of the file.

```
const ALBUM_PASSWORD = import.meta.env.VITE_ALBUM_PASSWORD || "";
const LOCKED_TRACKS_RAW = import.meta.env.VITE_LOCKED_TRACKS || "";
```

I then check if a track is locked by comparing the file names with each other.

```
// Check if a track is locked
function isLockedTrack(track) {
  const fileName = getFileName(track?.file);
  return LOCKED_TRACKS.includes(fileName);
}
```

If the track is locked and hasnt been unlocked yet then the UI hides the title and artist by replacing them with "???" here:

```
trackTitle.innerText = hideInfo ? "???" : track.title;
trackArtist.innerText = hideInfo ? "???" : track.artist;
```

Then when the player tries to select a locked track they are prompted to input a password

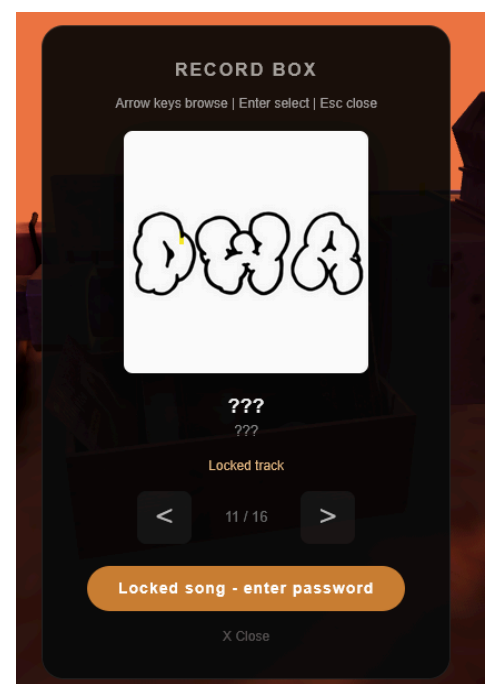
```
const input = window.prompt("Enter album password");
```

If the password is correct the tracks play as they should.

To handle the password i created a .env file which stores the album password and it stores the list of the locked tracks. This keeps it separate from the main code. I included the

 `.env.example`

file as well so someone else running the project knows what to set up without exposing the real password. I am aware that this is not fully secure because the password is still stored in the client side however for this project i thought it was a good way to show some features that i have learned from other modules as a way of implementing. Overall this system combines multiple parts of my project together like interaction, UI and audio.



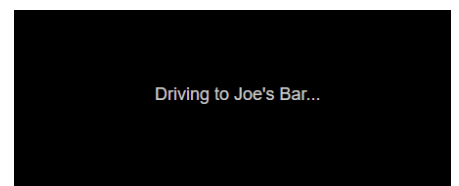
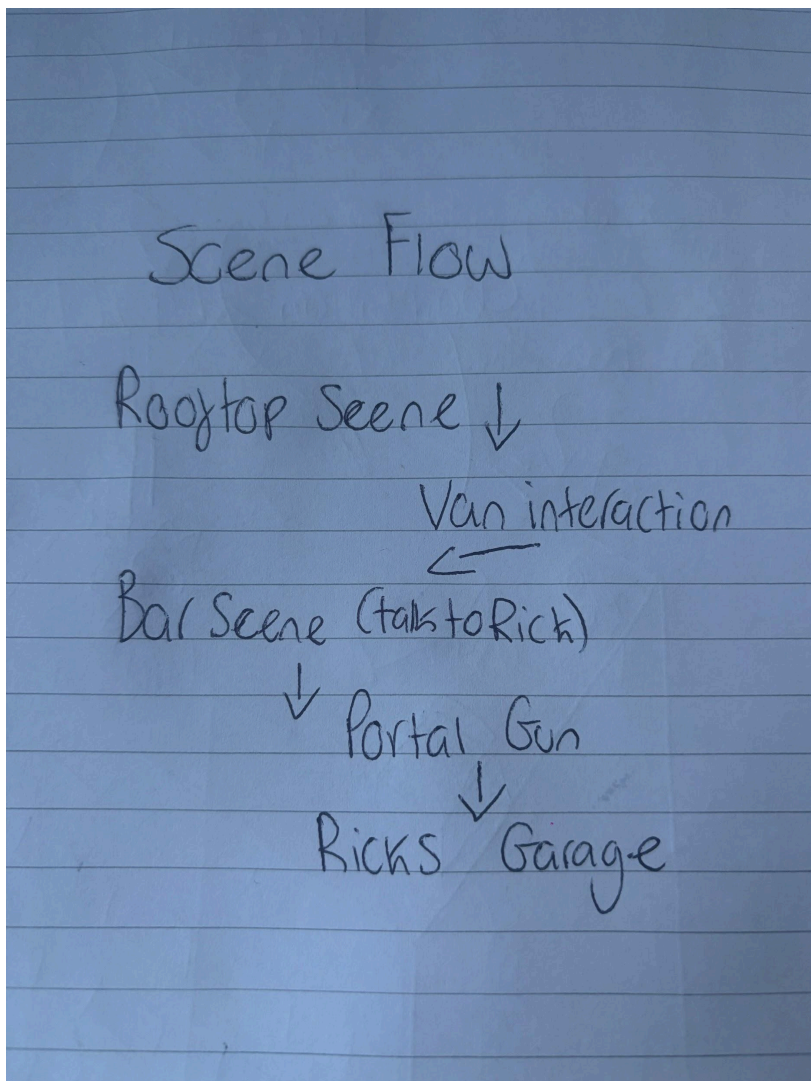
Scene Transitions

I wanted the transition between scenes to feel professional rather than just instantly switching. To get a decent looking transition between scenes i created the

JS `createTransition.js`

file, when the user interacts with a model like the van in my project it plays an engine sound, adds a small camera shake, fades the screen black and then shows a “driving to joes bar” message while the bar model loads in the background. Once its ready then it fades back in with the player already inside the bar spawning perfectly at the entrance door. The same system is used later for the portal gun transition into Ricks garage.

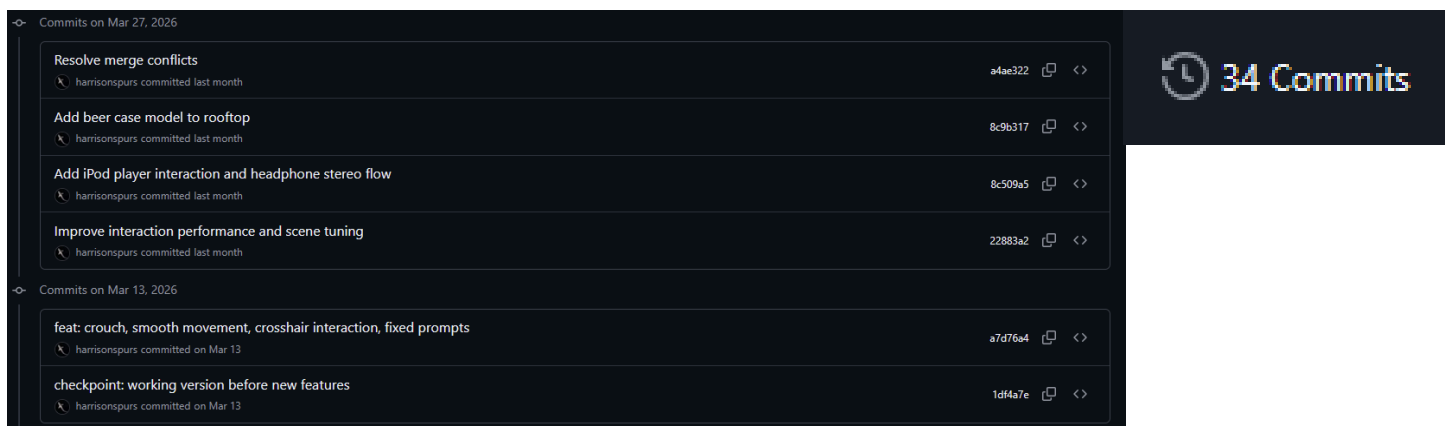
Instead of creating different separate Three.js scenes, I kept everything in one scene and just placed each environment at a different Y position so that basically each model just spawns underneath each other when loaded however you can not tell as a user because you are inside the 2 models that spawn after the rooftop scene . The rooftop is at $Y = 0$, the bar is at $Y = -200$, and the garage is at $Y = -250$. I then teleport the player between them using spawn points that are calculated from each model's bounding box. This made transitions much easier to manage and avoided having to reload everything each time.



Deployment and Github Version Control

Throughout the project I used GitHub for version control for committing regularly as new features were added and bugs were fixed. This helped me keep track of my progress and also gave me a safety net when testing new functions i was adding as I could always revert back if something broke.

To make the project accessible on my Oculus Quest without needing to run it locally, I deployed it using Vercel. This allowed me to open the experience directly in the Quest browser and test it properly in VR, rather than relying on desktop testing. Being able to test in the actual headset made a big difference, especially when working on movement, interaction and overall feel of the experience. Url link - <https://xrdesign.vercel.app/>



Characters and WebXR

In the bar scene when interacting with the animated model Rick Sanchez this makes him turn to face the player and open up a dialogue box where he asks you to find a portal gun. This then gives the player a clear objective instead of just walking around trying to figure out how the experience works. Once the portal gun is found my intention was for the user to see “press R to use portal gun” and immediately use the portal gun instead of handing it to Rick, this triggers the transition into the garage where Morty has his own dialogue. I also added a Pickle Rick model with a looping animation as a small detail near the end of the project.

For VR implementation i spent lots of time trying to get the VR version of my project actually working on my oculus quest, i spent time researching documentation like WebXR Device API Specification back and forth and also using AI like claude to help me pull code i need for my situation in the oculus quest browser. After research i enabled `renderer.xr` and used my floor reference in code so that the players height felt correct as i had an ongoing issue where you would spawn with your head stuck in the model. I also added an XR button that only shows if



```
// Add VR button only if XR is supported
if (navigator.xr) {
```

is available, so the project still works normally on desktop without an annoying VR button in the way

Getting VR working properly was one of the hardest parts of the project especially with limited time near the deadline. This is where I stopped implementing more into the story side of the project and put my full attention into making the experience work in VR. I did use AI such as Claude to help me understand parts of the WebXR setup and debug controller input issues, but I still had to integrate this into my own code, adjust it to match my player controller, and debug a lot of issues myself. I used it more as a support tool to help me move forward when I was stuck rather than something that built it for me.

On my Oculus Quest, I was able to get the scene loading and head tracking working correctly, and after further development I managed to get full movement and interaction working as well. The player can move using the controller input and interact with objects like the record box and boombox, which makes the experience feel much more complete and immersive in VR. This was challenging because my desktop setup uses PointerLockControls which doesn't match how WebXR handles movement and camera positioning. I had to work around this and adapt my system so that VR input could still control the player as intended.

Challenges and Problem Solving

One of the biggest challenges i faced was the physics system. I learnt that [Ammo.js](#) is loaded separately using WebAssembly so i had to wrap everything inside a physics loader before i could even use any physics. Getting this working correctly took a while.

Another issue was player spawning in the bar. The position kept getting overridden by other code that shouldnt have been tied in with bar spawn. I fixed this by creating a single

`getBarSpawnPoint()` function in `JS createBar.js` that calculates the correct position using the model's bounding box so that nothing else could interfere with it.

This error with prompt overlap issues happened because every object was creating its own UI without checking if something else was already on screen. This made it look completely

broken and glitchy. I fixed this by building `JS createPrompt.js` as a central system where each prompt has a priority, so only one shows at a time.

The record box caused a big performance problem. Just looking at it would drop the frame rate badly. I found out this was because raycasting was being done on the full model every frame. I fixed it by adding a simple invisible proxy box and only checking every 80ms instead of every frame. After that, I reused this approach across the rest of the project.

The Blender animation attempt didn't work out in the end, but it was still useful. I spent time learning rigging and animation, but every export either broke the model or didn't include

animations properly. Even though I didn't use it in the final project, it helped me understand how models and animations actually work together.

Reflection and Evaluation

I had a lot of fun building this project and it's easily one of my favourite things I have made on this course. From the image I pictured at the start of the project i didnt think that it would work as well as it has done, seeing this turn into a full interactive experience with music, characters and multiple scenes was rewarding. The music system in particular is one of my favourite parts of the project and stands out to me as it feels like almost a complete feature rather than a basic demo, the record box, boombox, headphones, glow effect and locked tracks all work together and are based on something personal which made it much more enjoyable for me to build and to show my friends when finished.

Throughout the project I learned a lot of new skills with Three.js. I improved my understanding of working with 3D models and formats like GLB, how physics systems like Ammo.js actually behave in an environment and how important it is to structure code properly. Splitting everything into separate components with their own update functions made it much easier to expand the project later on without breaking my code. I also learned a lot through fixing problems, like improving performance with raycasting and getting interactions to feel smooth and responsive.

Looking back there are definitely things I could have done better. I could have had the idea to start testing in VR much earlier instead of leaving it closer to the deadline, which would have made development a lot more smoother. Some parts of the project still rely on hardcoded values, and I could have planned systems like positioning more cleanly from the start as i did have a lot of issues with positioning my models. I also focused heavily on adding lots of features which makes me question whether I could have spent more time polishing what I already had instead of expanding so much, however i had enjoyed building and adding models so much that my mind kept on thinking about what else i could add and what other cool ideas i could implement instead of polishing what i already had

References

Three.js Documentation. Three.js Manual. Available at: <https://threejs.org/docs/>

W3C. WebXR Device API Specification. Available at: <https://www.w3.org/TR/webxr/>

Enable3D. @enable3d/ammo-physics. Available at: <https://enable3d.io>

Sketchfab. 3D Model Repository. Available at: <https://sketchfab.com>

Blender Foundation. Blender Manual. Available at: <https://docs.blender.org>

Stack Overflow. Developer Community. Available at: <https://stackoverflow.com>

